Table des matières

1 Code source

1.1 Game1.cs

Listing 1 – Game1.cs

```
using System;
using System.Collections.Generic;
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework.Input;

namespace santaClash;

public class Game1 : Game
{
    private GraphicsDeviceManager _graphics;
    private SpriteBatch _spriteBatch;
    private Point gameSize = new Point(1250, 1000);
    private Texture2D background;
    private Texture2D billet;
    private Texture2D leprechaun;
    private Texture2D santaCash;
    private Player player1;
    private Player player2;
    private Santa santa;

    // liste des ennemis (billets)
    private List<Enemy> enemies = new List<Enemy>();
    private Random random = new Random();
    private float spawnTimer = 0f;
    private float spawnInterval = 3.5f; // temps entre chaque vague
    private int enemiesPerWave = 4; // nombre de billets par vague

    // game over
    private bool gameOver = false;
    private SpriteFont font;
    private Texture2D pixel; // texture 1x1 pour dessiner les rectangles

    public Game1()
    {
        _graphics = new GraphicsDeviceManager(this);
        Content.RootDirectory = "Content";
        IsMouseVisible = true;
```

```
    }

    protected override void Initialize()
    {
        // TODO: Add your initialization logic here
        _graphics.PreferredBackBufferWidth = gameSize.X;
        _graphics.PreferredBackBufferHeight = gameSize.Y;
        _graphics.ApplyChanges();

        base.Initialize();
    }

    protected override void LoadContent()
    {
        _spriteBatch = new SpriteBatch(GraphicsDevice);
        background = Content.Load<Texture2D>("background");
        billet = Content.Load<Texture2D>("dessin-billet-banque-euro-02");
        leprechaun = Content.Load<Texture2D>("leprechaun");
        santaCash = Content.Load<Texture2D>("santaCash");
        font = Content.Load<SpriteFont>("SpriteFont");
        // TODO: use this.Content to load your game content here


        pixel = new Texture2D(GraphicsDevice, 1, 1);
        pixel.SetData(new[] { Color.White });

        player1 = new Player(_graphics, leprechaun, new Vector2(10, 10), new Vecto
        player2 = new Player(_graphics, leprechaun, new Vector2(10, 10), new Vecto

        santa = new Santa(_graphics, santaCash, new Vector2(200, 200), new Vector2


    }

    protected override void Update(GameTime gameTime)
    {
        var keyboardState = Keyboard.GetState();

        if (GamePad.GetState(PlayerIndex.One).Buttons.Back == ButtonState.Pressed
            Exit();



        GamePad.SetVibration(PlayerIndex.One, 0.5f, 0.5f);
        GamePad.SetVibration(PlayerIndex.Two, 0.5f, 0.5f);

        if (gameOver)
        {
            // appuyer sur R pour recommencer ou bouton Start/A sur manette
            GamePadState gamePad1 = GamePad.GetState(PlayerIndex.One);
            GamePadState gamePad2 = GamePad.GetState(PlayerIndex.Two);

            if (keyboardState.IsKeyDown(Keys.R) ||
```

```
                gamePad1.Buttons.Start == ButtonState.Pressed ||
                gamePad1.Buttons.A == ButtonState.Pressed ||
                gamePad2.Buttons.Start == ButtonState.Pressed ||
                gamePad2.Buttons.A == ButtonState.Pressed)
            {
                RestartGame();
            }


            base.Update(gameTime);
            return;
        }

        // TODO: Add your update logic here

        keyPressedPlayer();
        int screenWidth = GraphicsDevice.Viewport.Width;
        int screenHeight = GraphicsDevice.Viewport.Height;


        player1.Update(gameTime, screenWidth, screenHeight);
        player2.Update(gameTime, screenWidth, screenHeight);
        santa.Update();

        // spawn des ennemis
        spawnTimer += (float)gameTime.ElapsedGameTime.TotalSeconds;
        if (spawnTimer >= spawnInterval)
        {
            SpawnWave();
            enemiesPerWave++; // augmenter le nombre d'ennemis par vague
            spawnInterval = Math.Max(0.5f, spawnInterval - 0.1f);
            spawnTimer = 0f;
        }

        // update des ennemis
        foreach (var enemy in enemies)
        {
            enemy.Update(gameTime, santa.position);
        }

        // collisions joueurs avec ennemis
        CheckCollisions();

        // collision ennemis avec santa
        CheckSantaCollisions();

        if (santa.BarrePleine())
        {
            gameOver = true;
        }

        base.Update(gameTime);
    }
```

```
protected override void Draw(GameTime gameTime)
{
    GraphicsDevice.Clear(Color.CornflowerBlue);

    // TODO: Add your drawing code here
    _spriteBatch.Begin();
    _spriteBatch.Draw(background, new Rectangle(0, 0, gameSize.X, gameSize.Y),

    santa.Draw(_spriteBatch);
    player1.Draw(_spriteBatch);
    player2.Draw(_spriteBatch);

    // dessiner les ennemis
    foreach (var enemy in enemies)
    {
        enemy.Draw(_spriteBatch);
    }


    DrawMoneyBar();

    // dessiner les scores
    DrawScores();

    // dessiner game over
    if (gameOver)
    {
        DrawGameOver();
    }

    _spriteBatch.End();
    base.Draw(gameTime);
}


public void keyPressedPlayer()
{
    var keyboardState = Keyboard.GetState();


    GamePadState gamePad1 = GamePad.GetState(PlayerIndex.One);
    GamePadState gamePad2 = GamePad.GetState(PlayerIndex.Two);


    const float deadZone = 0.2f;

    // ========== PLAYER 1 ==========

    if (gamePad1.IsConnected)
    {
        Vector2 leftStick1 = gamePad1.ThumbSticks.Left;
```

```
        if (Math.Abs(leftStick1.X) > deadZone)
        {
            player1.position.X += leftStick1.X * player1.vitesse.X;
        }
        if (Math.Abs(leftStick1.Y) > deadZone)
        {

            player1.position.Y -= leftStick1.Y * player1.vitesse.Y;
        }


        if (gamePad1.DPad.Right == ButtonState.Pressed)
            player1.position.X += player1.vitesse.X;
        if (gamePad1.DPad.Left == ButtonState.Pressed)
            player1.position.X -= player1.vitesse.X;
        if (gamePad1.DPad.Up == ButtonState.Pressed)
            player1.position.Y -= player1.vitesse.Y;
        if (gamePad1.DPad.Down == ButtonState.Pressed)
            player1.position.Y += player1.vitesse.Y;
    }


    if (keyboardState.IsKeyDown(Keys.S) && keyboardState.IsKeyDown(Keys.D))
    {
        player1.position.Y += player1.vitesse.Y;
        player1.position.X += player1.vitesse.X;
    }
    else if (keyboardState.IsKeyDown(Keys.S) && keyboardState.IsKeyDown(Keys.A
    {
        player1.position.Y += player1.vitesse.Y;
        player1.position.X -= player1.vitesse.X;
    }
    else if (keyboardState.IsKeyDown(Keys.W) && keyboardState.IsKeyDown(Keys.D
    {
        player1.position.Y -= player1.vitesse.Y;
        player1.position.X += player1.vitesse.X;
    }
    else if (keyboardState.IsKeyDown(Keys.W) && keyboardState.IsKeyDown(Keys.A
    {
        player1.position.Y -= player1.vitesse.Y;
        player1.position.X -= player1.vitesse.X;
    }
    else if (keyboardState.IsKeyDown(Keys.D))
    {
        player1.position.X += player1.vitesse.X;
    }
    else if (keyboardState.IsKeyDown(Keys.A))
    {
        player1.position.X -= player1.vitesse.X;
    }
    else if (keyboardState.IsKeyDown(Keys.W))
    {
```

```
        player1.position.Y -= player1.vitesse.Y;
    }
    else if (keyboardState.IsKeyDown(Keys.S))
    {
        player1.position.Y += player1.vitesse.Y;
    }

    // ========== PLAYER 2 ==========

    if (gamePad2.IsConnected)
    {
        Vector2 leftStick2 = gamePad2.ThumbSticks.Left;


        if (Math.Abs(leftStick2.X) > deadZone)
        {
            player2.position.X += leftStick2.X * player2.vitesse.X;
        }
        if (Math.Abs(leftStick2.Y) > deadZone)
        {

            player2.position.Y -= leftStick2.Y * player2.vitesse.Y;
        }


        if (gamePad2.DPad.Right == ButtonState.Pressed)
            player2.position.X += player2.vitesse.X;
        if (gamePad2.DPad.Left == ButtonState.Pressed)
            player2.position.X -= player2.vitesse.X;
        if (gamePad2.DPad.Up == ButtonState.Pressed)
            player2.position.Y -= player2.vitesse.Y;
        if (gamePad2.DPad.Down == ButtonState.Pressed)
            player2.position.Y += player2.vitesse.Y;
    }


    if (keyboardState.IsKeyDown(Keys.Up) && keyboardState.IsKeyDown(Keys.Right
    {
        player2.position.Y -= player2.vitesse.Y;
        player2.position.X += player2.vitesse.X;
    }
    else if (keyboardState.IsKeyDown(Keys.Up) && keyboardState.IsKeyDown(Keys.
    {
        player2.position.Y -= player2.vitesse.Y;
        player2.position.X -= player2.vitesse.X;
    }
    else if (keyboardState.IsKeyDown(Keys.Down) && keyboardState.IsKeyDown(Keys
    {
        player2.position.Y += player2.vitesse.Y;
        player2.position.X += player2.vitesse.X;
    }
    else if (keyboardState.IsKeyDown(Keys.Down) && keyboardState.IsKeyDown(Keys
    {
```

```
            player2.position.Y += player2.vitesse.Y;
            player2.position.X -= player2.vitesse.X;
        }
        else if (keyboardState.IsKeyDown(Keys.Right))
        {
            player2.position.X += player2.vitesse.X;
        }
        else if (keyboardState.IsKeyDown(Keys.Left))
        {
            player2.position.X -= player2.vitesse.X;
        }
        else if (keyboardState.IsKeyDown(Keys.Down))
        {
            player2.position.Y += player2.vitesse.Y;
        }
        else if (keyboardState.IsKeyDown(Keys.Up))
        {
            player2.position.Y -= player2.vitesse.Y;
        }
    }

    private void SpawnWave()
    {

        for (int i = 0; i < enemiesPerWave; i++)
        {

            Vector2 spawnPos = GetRandomSpawnPosition();
            Enemy enemy = new Enemy(_graphics, billet, new Vector2(2, 2), spawnPos
            enemies.Add(enemy);

        }
    }

    private Vector2 GetRandomSpawnPosition()
    {
        int side = random.Next(4); // 0=haut, 1=bas, 2=gauche, 3=droite
        float x = 0;
        float y = 0;

        switch (side)
        {
            case 0: // haut
                x = random.Next(0, gameSize.X);
                y = -50;
                break;
            case 1: // bas
                x = random.Next(0, gameSize.X);
                y = gameSize.Y + 50;
                break;
            case 2: // gauche
                x = -50;
                y = random.Next(0, gameSize.Y);
```

```
                break;
            case 3: // droite
                x = gameSize.X + 50;
                y = random.Next(0, gameSize.Y);
                break;
        }
        return new Vector2(x, y);
    }

    private void CheckCollisions()
    {
        Rectangle player1Bounds = player1.GetBounds();
        Rectangle player2Bounds = player2.GetBounds();

        foreach (var enemy in enemies)
        {
            if (!enemy.isAlive) continue;

            Rectangle enemyBounds = enemy.GetBounds();

            // collision avec player1
            if (player1Bounds.Intersects(enemyBounds))
            {
                enemy.isAlive = false;
                player1.score++;
            }
            // collision avec player2
            else if (player2Bounds.Intersects(enemyBounds))
            {
                enemy.isAlive = false;
                player2.score++;
            }
        }

        // nettoyer les ennemis morts
        enemies.RemoveAll(e => !e.isAlive);
    }

    private void CheckSantaCollisions()
    {
        Rectangle santaBounds = santa.GetBounds();

        foreach (var enemy in enemies)
        {
            if (!enemy.isAlive) continue;

            Rectangle enemyBounds = enemy.GetBounds();

            if (santaBounds.Intersects(enemyBounds))
            {
                enemy.isAlive = false;
                santa.AjouterArgent(10f); // augmente la barre d'argent
            }
```

```
        }

        // nettoyer les ennemis morts
        enemies.RemoveAll(e => !e.isAlive);
    }

    private void DrawMoneyBar()
    {
        int barX = gameSize.X / 2 - 150;
        int barY = 20;
        int barWidth = 300;
        int barHeight = 30;

        // bordure noire
        _spriteBatch.Draw(pixel, new Rectangle(barX - 3, barY - 3, barWidth + 6, b


        _spriteBatch.Draw(pixel, new Rectangle(barX, barY, barWidth, barHeight), C


        int fillWidth = (int)(barWidth * (santa.argentActuel / santa.argentMax));
        float ratio = santa.argentActuel / santa.argentMax;
        Color barColor = new Color(
            (int)(255 * ratio),      // rouge augmente
            (int)(100 * (1 - ratio)), // vert diminue
            0
        );
        _spriteBatch.Draw(pixel, new Rectangle(barX, barY, fillWidth , barHeight),


        _spriteBatch.Draw(billet, new Rectangle(barX - 60, barY - 10, 50, 50), Col
    }

    private void DrawScores()
    {
        int scoreBoxWidth = 200;
        int scoreBoxHeight = 50;
        int margin = 20;


        int p1X = margin;
        int p1Y = gameSize.Y - scoreBoxHeight - margin;


        _spriteBatch.Draw(pixel, new Rectangle(p1X - 2, p1Y - 2, scoreBoxWidth + 4
        _spriteBatch.Draw(pixel, new Rectangle(p1X, p1Y, scoreBoxWidth, scoreBoxHe


        int p1ScoreWidth = Math.Min(player1.score * 10, scoreBoxWidth - 10);
        _spriteBatch.Draw(pixel, new Rectangle(p1X + 5, p1Y + 5, p1ScoreWidth, sco

        int p2X = gameSize.X - scoreBoxWidth - margin;
        int p2Y = gameSize.Y - scoreBoxHeight - margin;
```

```
        _spriteBatch.Draw(pixel, new Rectangle(p2X - 2, p2Y - 2, scoreBoxWidth + 4
        _spriteBatch.Draw(pixel, new Rectangle(p2X, p2Y, scoreBoxWidth, scoreBoxHe

        int p2ScoreWidth = Math.Min(player2.score * 10, scoreBoxWidth - 10);
        _spriteBatch.Draw(pixel, new Rectangle(p2X + 5, p2Y + 5, p2ScoreWidth, sco
    }

    private void DrawGameOver()
    {

        _spriteBatch.Draw(
            pixel,
            new Rectangle(0, 0, gameSize.X, gameSize.Y),
            new Color(0, 0, 0, 200)
        );

        int centerX = gameSize.X / 2;
        int centerY = gameSize.Y / 2;

        int panelWidth = 480;
        int panelHeight = 300;
        int panelX = centerX - panelWidth / 2;
        int panelY = centerY - panelHeight / 2;

        // OMBRE DU PANNEAU
        _spriteBatch.Draw(
            pixel,
            new Rectangle(panelX + 6, panelY + 6, panelWidth, panelHeight),
            new Color(0, 0, 0, 120)
        );

        // PANNEAU PRINCIPAL
        _spriteBatch.Draw(
            pixel,
            new Rectangle(panelX, panelY, panelWidth, panelHeight),
            new Color(25, 25, 35)
        );

        // BORDURE
        _spriteBatch.Draw(
            pixel,
            new Rectangle(panelX - 3, panelY - 3, panelWidth + 6, panelHeight + 6)
            Color.Black
        );

        // TITRE
        string title = "GAME OVER";
        Vector2 titleSize = font.MeasureString(title);
        _spriteBatch.DrawString(
            font,
            title,
```

```
            new Vector2(centerX - titleSize.X / 2, panelY + 25),
            Color.OrangeRed
        );

        // LIGNE SEPARATION
        _spriteBatch.Draw(
            pixel,
            new Rectangle(panelX + 40, panelY + 80, panelWidth - 80, 2),
            Color.Black
        );

        // SCORES
        string score1 = $"PLAYER 1  :  {player1.score}";
        string score2 = $"PLAYER 2  :  {player2.score}";

        Vector2 s1Size = font.MeasureString(score1);
        Vector2 s2Size = font.MeasureString(score2);

        _spriteBatch.DrawString(
            font,
            score1,
            new Vector2(centerX - s1Size.X / 2, panelY + 120),
            Color.White
        );

        _spriteBatch.DrawString(
            font,
            score2,
            new Vector2(centerX - s2Size.X / 2, panelY + 160),
            Color.White
        );

        // TEXTE ACTION
        string action = "Press R or Start to Restart";
        Vector2 actionSize = font.MeasureString(action);

        _spriteBatch.DrawString(
            font,
            action,
            new Vector2(centerX - actionSize.X / 2, panelY + panelHeight - 50),
            Color.LightGray
        );
    }


    private void RestartGame()
    {
        gameOver = false;
        santa.argentActuel = 0f;
        player1.score = 0;
        player2.score = 0;
        player1.position = new Vector2(350, 465);
        player2.position = new Vector2(850, 465);
```

```
            enemies.Clear();
            spawnTimer = 0f;
            enemiesPerWave = 4;
        }
    }
```

1.2 Enemy.cs

Listing 2 – Enemy.cs

```
using System;
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework.Input;

namespace santaClash;

public class Enemy : GameObject
{

    private readonly GraphicsDeviceManager _graphics;
    private Texture2D _texture;

    private float scale = 0.2f;
    private float speed = 2f;

    public Enemy(GraphicsDeviceManager graphics, Texture2D texture, Vector2 vitesse
    {
        _graphics = graphics;
        _texture = texture;

    }

    public void Update(GameTime gameTime, Vector2 santaPosition)
    {
        if (!isAlive) return;

        base.Update(gameTime);


        Vector2 direction = santaPosition - position;
        if (direction != Vector2.Zero)
            direction.Normalize();

        position += direction * speed;
    }

    public Rectangle GetBounds()
    {
        return new Rectangle((int)position.X, (int)position.Y, (int)(_texture.Width
    }

    public override void Draw(SpriteBatch spriteBatch)
    {
        if (!isAlive) return;
```

```
        // spriteBatch.Draw(_texture, position, Color.White, scale);
        //null signifie qu'on prend toute l'image source
        // 0f = Pas de rotation
        //Vector2.Zero = Origine en haut a gauche
        //SpriteEffects.None pas d effet miroir
        // 0f = couche de profondeur
        spriteBatch.Draw(_texture, position, null, Color.White, 0f, Vector2.Zero, 
    }

}
```

1.3 Player.cs

Listing 3 – Player.cs

```
using System;
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework.Input;

namespace santaClash;

public class Player : GameObject
{
    private readonly GraphicsDeviceManager _graphics;
    private Texture2D _texture;

    private float scale = 0.6f;
    public int score = 0;

    public Player(GraphicsDeviceManager graphics, Texture2D texture, Vector2 vitess
    {
        _graphics = graphics;
        _texture = texture;

    }

    public void Update(GameTime gameTime, int screenWidth, int screenHeight)
    {
        base.Update(gameTime);
        float spriteWidth = _texture.Width * scale;
        float spriteHeight = _texture.Height * scale;

        float maxX = screenWidth - spriteWidth;
        float maxY = screenHeight - spriteHeight;

        position.X = MathHelper.Clamp(position.X, 0, maxX);
        position.Y = MathHelper.Clamp(position.Y, 0, maxY);
    }

    public Rectangle GetBounds()
    {
        return new Rectangle((int)position.X, (int)position.Y, (int)(_texture.Width
    }
```

```
    public override void Draw(SpriteBatch spriteBatch)
    {
        // spriteBatch.Draw(_texture, position, Color.White, scale);
        //null signifie qu'on prend toute l'image source
        // 0f = Pas de rotation
        //Vector2.Zero = Origine en haut a gauche
        //SpriteEffects.None pas d effet miroir
        // 0f = couche de profondeur
        spriteBatch.Draw(_texture, position, null, Color.White, 0f, Vector2.Zero, s
    }


}
```

1.4 Santa.cs

Listing 4 – Santa.cs

```
using System;
using System.Diagnostics;
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework.Input;

namespace santaClash;

public class Santa : GameObject
{
    private readonly GraphicsDeviceManager _graphics;
    private Vector2 _position;
    private Vector2 _velocity;
    private Texture2D _texture;
    private readonly Vector2 _targetPosition;
    private readonly Random _random = new Random();

    private Vector2 _origin;

    private float scale = 0.4f;
    private float _attractionStrength = 50f;
    private float _noiseStrength = 20f;

    // barre d'argent
    public float argentMax = 100f;
    public float argentActuel = 0f;

    public Santa(GraphicsDeviceManager graphics, Texture2D texture, Vector2 vitesse
    {
        _graphics = graphics;
        _texture = texture;
        _origin = new Vector2(_texture.Width / 2f, _texture.Height / 2f);
        _position = position;
        _velocity = vitesse;
        _targetPosition = new Vector2(graphics.PreferredBackBufferWidth / 2f, grap
```

```
    }


    public void Update()
    {

        Vector2 toCenter = _targetPosition - _position;
        if (toCenter != Vector2.Zero)
            toCenter.Normalize();

        float noiseX = (float)(_random.NextDouble() - 10);
        float noiseY = (float)(_random.NextDouble() - 10);
        Vector2 noise = new Vector2(noiseX, noiseY);
        if (noise != Vector2.Zero)
            noise.Normalize();


        Vector2 acceleration = toCenter * _attractionStrength + noise * _noiseStren
        Debug.WriteLine("Acceleration: " + acceleration);

        _velocity += acceleration;

        _position += _velocity;
    }

    public Rectangle GetBounds()
    {
        return new Rectangle((int)(position.X - _origin.X * scale), (int)(position
    }

    public void AjouterArgent(float montant)
    {
        argentActuel += montant;
        if (argentActuel > argentMax)
            argentActuel = argentMax;
    }

    public bool BarrePleine()
    {
        return argentActuel >= argentMax;
    }

    public override void Draw(SpriteBatch spriteBatch)
    {
        // spriteBatch.Draw(_texture, position, Color.White, scale);
        //null signifie qu'on prend toute l'image source
        // 0f = Pas de rotation
        //Vector2.Zero = Origine en haut a gauche
        //SpriteEffects.None pas d effet miroir
        // 0f = couche de profondeur
        spriteBatch.Draw(_texture, position, null, Color.White, 0f, _origin, scale
    }
}
```